```
                                        if STATE.breach_stage_timer >= 180:
            STATE.breach_stage = 2
            STATE.breach_stage_timer = 0
    elif STATE.breach_stage == 2:
        if STATE.breach_stage_timer == 1:
            trigger_radio_chatter(
                "GENERAL: "BRAVO! They're breaching the bay doors!""
            )
        if STATE.breach_stage_timer >= 180:
            STATE.breach_stage = 3
            STATE.breach_stage_timer = 0
    elif STATE.breach_stage == 3:
        if STATE.breach_stage_timer % 15 == 0:
            audio.play("door_rattle", volume=0.5)
            STATE.screen_shake = max(STATE.screen_shake, 5)
        if STATE.breach_stage_timer >= 300:
            STATE.breach_stage = 4
            STATE.breach_stage_timer = 0
    elif STATE.breach_stage == 4:
        STATE.screen_shake = max(STATE.screen_shake, 8)
        if STATE.breach_stage_timer % 4 == 0 and STATE.player:
            for _ in range(3):
                px = 34 * TILE_SIZE + random.randint(-3 * TILE_SIZE, 3 * TILE_SIZE)
                py = 58 * TILE_SIZE + random.randint(-TILE_SIZE, 0)
                p = Particle(px, py, random.choice([(140, 130, 120), (90, 85, 70)]))
                p.life = random.randint(20, 40)
                p.vy = random.uniform(-0.5, 0.5)
                p.vx = random.uniform(-0.5, 0.5)
                STATE.particles.append(p)
        if STATE.breach_stage_timer >= 180:
            STATE.breach_stage = 5
            STATE.breach_stage_timer = 0
    elif STATE.breach_stage == 5:
        STATE.gl_flash_timer = 30
        effects.shake(20)
        STATE.bay_spawn_done = True
        STATE.breach_sequence_active = False
        # ================================
        # OPEN BAY DOORS
        # ================================
        for d in STATE.doors:
            if d.is_bay:
                d.locked = False
                d.target_open = True
                d.anim_progress = 1.0
                d.collision_disabled = True
                d.manual_timer = 9999
        # ================================
        # OPEN THE GATE (row 31) so rear enemies pour through
        # ================================
        for d in STATE.doors:
            if getattr(d, "is_gate", False):
                d.locked = False
                d.target_open = True
                d.anim_progress = 1.0
                d.collision_disabled = True
                d.manual_timer = 9999
        # ================================
        # 8 ENEMIES FROM BAY DOORS (south side, row 58-59)
        # ================================
        bay_spawns = [
            _ts(30, 60), _ts(32, 60), _ts(35, 60), _ts(38, 60),
            _ts(31, 60), _ts(33, 61), _ts(35, 61), _ts(38, 61),
        ]
        bay_targets = [
            _ts(30, 50), _ts(32, 50), _ts(36, 50), _ts(38, 50),
            _ts(31, 57), _ts(33, 57), _ts(37, 57), _ts(33, 54),
        ]
        for i, (sx, sy) in enumerate(bay_spawns):
            tx, ty = bay_targets[i % len(bay_targets)]
            new_e = Enemy(sx, sy)
            new_e.alerted = True
            new_e.state = "CHASE"
            new_e.last_seen_pos = (tx, ty)
            new_e.is_ambush_enemy = True
            new_e.is_breach_enemy = True
            new_e.fire_weakness = 1.35
            new_e.hp = 135
            new_e.fire_cooldown = 60 + i * 15 + random.randint(0, 30)
            new_e.speed = random.uniform(2.0, 2.7)
            STATE.enemies.append(new_e)
            for _ in range(10):
                STATE.particles.append(Particle(sx, sy, (150, 150, 150)))
            for _ in range(10):
                STATE.particles.append(Particle(sx, sy, (255, 160, 50)))
        # ================================
        # 6 ENEMIES FROM COMMANDER ROOM (rows 31-34)
        # ================================
        rear_spawns = [
            _ts(38, 33), _ts(40, 33),
            _ts(4,  33), _ts(40, 33),
            _ts(38, 33), _ts(35, 33),
        ]
        rear_targets = [
            _ts(38, 37), _ts(40, 37),
            _ts(8,  34), _ts(40, 34),
            _ts(34, 40), _ts(34, 40),
        ]
        for i, (sx, sy) in enumerate(rear_spawns):
            tx, ty = rear_targets[i % len(rear_targets)]
            new_e = Enemy(sx, sy)
            new_e.alerted = True
            new_e.state = "CHASE"
            new_e.last_seen_pos = (tx, ty)
            new_e.is_ambush_enemy = True
            new_e.is_breach_enemy = True
            new_e.fire_weakness = 1.35
            new_e.hp = 135
            new_e.fire_cooldown = 40 + i * 20 + random.randint(0, 20)
            new_e.speed = random.uniform(2.0, 2.6)
            STATE.enemies.append(new_e)
            for _ in range(15):
                STATE.particles.append(Particle(sx, sy, (200, 200, 200)))
        # ================================
        # EFFECTS
        # ================================
        audio.play("breach", volume=0.9)
        audio.play("explosion", volume=0.7)
        STATE.floating_texts.append([
            35 * TILE_SIZE + 16, 55 * TILE_SIZE,
            "BAY DOOR BREACH!",
            (255, 100, 50), 180,
        ])
        STATE.floating_texts.append([
            35 * TILE_SIZE + 16, 31 * TILE_SIZE,
            "COMMAND ROOM BREACH!",
            (255, 70, 40), 180,
        ])
        trigger_radio_chatter(
            "GENERAL: "They're coming from both sides! Get to the truck NOW!"",
            color=(255, 80, 60),
            duration=300,
        )
        STATE.breach_stage = 6
# --- Window escape animation ---
if STATE.window_escape_timer > 0:
    STATE.window_escape_timer -= 1
    bay_door_y = 58 * TILE_SIZE + 16
    if STATE.window_escape_pos and not STATE.window_escaped:
        dx = 35 * TILE_SIZE + 16 - STATE.window_escape_pos[0]
        dy = bay_door_y - STATE.window_escape_pos[1]
        dist = math.hypot(dx, dy)
        if dist > 5:
            STATE.window_escape_pos[0] += (dx / dist) * 3.5
            STATE.window_escape_pos[1] += (dy / dist) * 3.5
        else:
            STATE.window_escaped = True
    if STATE.commander_escape_pos and not STATE.commander_escaped:
        dx = 33 * TILE_SIZE + 16 - STATE.commander_escape_pos[0]
        dy = bay_door_y - STATE.commander_escape_pos[1]
        dist = math.hypot(dx, dy)
        if dist > 5:
            STATE.commander_escape_pos[0] += (dx / dist) * 3.5
            STATE.commander_escape_pos[1] += (dy / dist) * 3.5
        else:
            STATE.commander_escaped = True
    if STATE.window_escape_timer == 90:
        trigger_radio_chatter(
            "GENERAL: "Window escaped! Get that sniveler and get out!""
        )
    if STATE.window_escape_timer == 0:
        STATE.window_escaped = True
        STATE.commander_escaped = True
solid_walls = STATE.walls[:]
for d in STATE.doors:
    if d.collision_rect:
        solid_walls.append(d.collision_rect)
gate_locked = any(d.locked for d in STATE.doors if getattr(d, "is_gate", False))
if gate_locked and STATE.player:
    dist_to_console = math.hypot(
        STATE.player.rect.centerx - (33 * TILE_SIZE + 16),
        STATE.player.rect.centery - (31 * TILE_SIZE + 16),
    )
    if dist_to_console < 100:
        if STATE.has_keycard:
            # Keycard bypass — instant open, no minigame
            if keys[pygame.K_e]:
                for d in STATE.doors:
                    if getattr(d, "is_gate", False):
                        d.locked = False
                        d.target_open = True
                        audio.play("door_open", volume=0.5)
                        STATE.floating_texts.append(
                            [
                                d.rect.centerx,
                                d.rect.centery - 30,
                                "GATE OPENED — KEYCARD",
                                C_YELLOW,
                                120,
                            ]
                        )
                STATE.gate_hacking = False
                STATE.gate_hack_progress = 0.0
        else:
            # No keycard — hack minigame
            if keys[pygame.K_e]:
                STATE.gate_hacking = True
                STATE.gate_hack_progress += 1.0 / 180.0
                if (pygame.time.get_ticks() // 200) % 2 == 0:
                    audio.play("hack_beep", volume=0.3)
                if STATE.gate_hack_progress >= 1.0:
                    for d in STATE.doors:
                        if getattr(d, "is_gate", False):
                            d.locked = False
                            d.target_open = True
                            audio.play("door_open", volume=0.5)
                            STATE.floating_texts.append(
                                [
                                    d.rect.centerx,
                                    d.rect.centery - 30,
                                    "GATE HACKED!",
                                    C_YELLOW,
                                    120,
                                ]
                            )
                    STATE.gate_hacking = False
                    STATE.gate_hack_progress = 0.0
            else:
                STATE.gate_hacking = False
                STATE.gate_hack_progress = 0.0
    else:
        if STATE.gate_hacking:
            STATE.gate_hacking = False
            STATE.gate_hack_progress = 0.0
else:
    if STATE.gate_hacking:
        STATE.gate_hacking = False
        STATE.gate_hack_progress = 0.0
for b in STATE.thrown_bottles[:]:
    prev_x, prev_y = b["x"], b["y"]
    b["t"] = min(1.0, b["t"] + b["speed"])
    new_x = b["start_x"] + (b["tx"] - b["start_x"]) * b["t"]
    new_y = b["start_y"] + (b["ty"] - b["start_y"]) * b["t"]
    hit_wall = False
    impact_x, impact_y = new_x, new_y
    # Molotov cannot pass through STATE.walls / closed STATE.doors
    for wall in solid_walls:
        clipped = wall.clipline((prev_x, prev_y), (new_x, new_y))
        if clipped:
            impact_x, impact_y = clipped[0]
            hit_wall = True
            break
    if b["t"] >= 1.0 or hit_wall:
        _spawn_molotov_fire(impact_x, impact_y, solid_walls)
        STATE.thrown_bottles.remove(b)
    else:
        b["x"] = new_x
        b["y"] = new_y
# --- Molotov fire damage ---
for fire in STATE.molotov_fires[:]:
    fire["life"] -= 1
    fx, fy = fire["x"], fire["y"]
    if random.random() < 0.75:
        px = fx + random.uniform(-MOLOTOV_RADIUS * 0.55, MOLOTOV_RADIUS * 0.55)
        py = fy + random.uniform(-MOLOTOV_RADIUS * 0.55, MOLOTOV_RADIUS * 0.55)
        if math.hypot(px - fx, py - fy) < MOLOTOV_RADIUS * 0.65:
            col = random.choice([(255, 70, 20), (255, 150, 40), (255, 220, 90), (80, 70, 60)])
            p = Particle(px, py, col)
            p.vy -= random.uniform(0.6, 1.6)
            p.vx *= 0.35
            p.life = random.randint(18, 34)
            STATE.particles.append(p)
    for e in STATE.enemies:
        edist = math.hypot(e.rect.centerx - fx, e.rect.centery - fy)
        if edist < MOLOTOV_RADIUS and not _wall_blocks_line(
            fx, fy, e.rect.centerx, e.rect.centery, solid_walls
        ):
            heat = 1.0 - edist / MOLOTOV_RADIUS
            fire_mult = getattr(e, "fire_weakness", 1.0)
            e.hp -= MOLOTOV_ENEMY_DAMAGE * (0.45 + heat) * fire_mult
            if getattr(e, "is_breach_enemy", False) and random.random() < 0.008:
                STATE.particles.append(Particle(e.rect.centerx, e.rect.centery, (255, 200, 70)))
            e.alerted = True
            e.combat_alert_timer = max(getattr(e, "combat_alert_timer", 0), 180)
            # Fire should alert enemies, but it should NOT constantly overwrite
            # their target if they can already see the player.
            if STATE.player:
                plx = STATE.player.rect.centerx
                ply = STATE.player.rect.centery
                can_see_player = not _wall_blocks_line(
                    e.rect.centerx,
                    e.rect.centery,
                    plx,
                    ply,
                    solid_walls,
                )
            else:
                can_see_player = False
            if can_see_player:
                e.last_seen_pos = (plx, ply)
                # Don't constantly yank enemies out of ATTACK flow.
                if e.state not in ("ATTACK", "CHASE"):
                    e.state = "CHASE"
            else:
                # Fire is only a distraction if they are not already committed.
                if e.state in ("IDLE", "PATROL"):
                    e.state = "SEARCH"
                    e.distraction_target = (fx, fy)
                    e.search_timer = max(getattr(e, "search_timer", 0), 120)
            if random.random() < 0.03:
                STATE.particles.append(
                    Particle(e.rect.centerx, e.rect.centery, (255, 110, 40))
                )
    if STATE.player and not STATE.player_dead:
        pdist = math.hypot(STATE.player.rect.centerx - fx, STATE.player.rect.centery - fy)
        player_burn_radius = MOLOTOV_RADIUS * 0.72
        if pdist < player_burn_radius and not _wall_blocks_line(fx, fy, STATE.player.rect.centerx, STATE.player.rect.centery, solid_walls):
            heat = 1.0 - pdist / player_burn_radius
            # BUG8 fix 4: route through take_damage() so i-frames apply.
            # Molotov burns tick every frame; without i-frames the player
            # could lose 100% HP in a single step into a fire zone.
            if STATE.player.take_damage(MOLOTOV_PLAYER_DAMAGE * (0.45 + heat)):
                player_hit_this_frame = True
    if fire["life"] <= 0:
        STATE.molotov_fires.remove(fire)
# Discard active fire zones
if STATE.player and not STATE.player_dead:
    STATE.fire_zones[:] = [fz for fz in STATE.fire_zones if fz.update(
        STATE.enemies, STATE.player.rect, audio
    )]
if STATE.mission_phase == "escape" and STATE.player:
    dist_to_truck = math.hypot(
        STATE.player.rect.centerx - STATE.extra_pos[0],
        STATE.player.rect.centery - STATE.extra_pos[1],
    )
    if dist_to_truck < 100:
        if STATE.enemies:
            if keys[pygame.K_e] and not STATE.hotwire_radio_said:
                STATE.hotwire_radio_said = True
                trigger_radio_chatter("GENERAL: "Not yet. Clear the yard before you put your head under that dash."")
            STATE.truck_hotwiring = False
            STATE.hotwire_selected_wire = None
        elif not STATE.truck_hotwiring:
            if keys[pygame.K_e]:
                STATE.truck_hotwiring = True
                _reset_hotwire_minigame()
                audio.play("door_creak", volume=0.3)
                if not STATE.hotwire_radio_said:
                    STATE.hotwire_radio_said = True
                    trigger_radio_chatter("GENERAL: "Open the panel. Red feeds yellow. Blue wakes green. Black kills white."")
        else:
            STATE.hotwire_panel_timer = min(30, STATE.hotwire_panel_timer + 1)
            if STATE.hotwire_spark_timer > 0:
                STATE.hotwire_spark_timer -= 1
    else:
        STATE.truck_hotwiring = False
        STATE.hotwire_stage = 0
        STATE.hotwire_selected_wire = None
        STATE.hotwire_panel_timer = 0
    if STATE.hotwire_fail_flash > 0:
        STATE.hotwire_fail_flash -= 1
if STATE.player:
    if STATE.player.hp <= 0 and not STATE.player_dead:
        # Death animation (Option B + Standard) handles the full sequence:
        # slow-mo, camera collapse, ragdoll, blood pool, finishes, radio cut.
        # Replaces the legacy single-burst + shake trigger.
        import death_anim
        death_anim.trigger_death()
    if STATE.player_dead:
        # update_death_anim() advances all 4 layers and returns the
        # number of game-logic ticks to run (slow-mo via accumulator).
        import death_anim
        ticks = death_anim.update_death_anim()
        # During slow-mo we want bullets/enemies to advance at the slowed
        # rate. The caller's existing per-frame logic runs once per tick;
        # we stash the tick count on STATE so the rest of game_update can
        # read it without restructuring the whole function.
        STATE._death_logic_ticks = ticks
    if not STATE.player_dead:
        STATE._death_logic_ticks = 1
    if not STATE.player_dead:
        dx, dy = 0, 0
        if not STATE.truck_hotwiring:
            # Delta-time aware: speed is in px/frame (e.g. 5), so
            # multiply by (60 * dt) to get px this frame at any FPS.
            # At 60 FPS dt=1/60, so 60*dt = 1 → identical to old behavior.
            _dt_mul = 60.0 * STATE.dt
            if keys[pygame.K_w]:
                dy -= STATE.player.speed * _dt_mul
            if keys[pygame.K_s]:
                dy += STATE.player.speed * _dt_mul
            if keys[pygame.K_a]:
                dx -= STATE.player.speed * _dt_mul
            if keys[pygame.K_d]:
                dx += STATE.player.speed * _dt_mul
        if dx != 0 and dy != 0:
            dx *= 0.707
            dy *= 0.707
        is_moving = dx != 0 or dy != 0
        STATE.player.move(dx, dy, solid_walls)
        if is_moving:
            STATE._footstep_timer -= 1
            if STATE._footstep_timer <= 0:
                audio.play("footstep", volume=0.2)
                STATE._footstep_timer = 15 + random.randint(0, 3)
        else:
            STATE._footstep_timer = 0
# Unified camera: Early + finish-frame snap + deadzone + clamp + recoil kick + shake.
# BOTH directions now use this same code path so the feel of "the camera
# following the player" cannot drift between them.
if STATE.player:
    fcx, fcy = update_camera(
        (STATE.player.rect.centerx, STATE.player.rect.centery),
        map_w_px, map_h_px,
    )
else:
    fcx, fcy = STATE.camera_x, STATE.camera_y
if STATE.shot_cooldown > 0:
    STATE.shot_cooldown -= 1
if STATE.is_reloading:
    STATE.reload_timer -= 1
    if STATE.reload_timer <= 0:
        STATE.ammo_in_mag = STATE.mag_size
        STATE.spare_mags -= 1
        STATE.is_reloading = False
if STATE.player:
    for d in STATE.doors:
        d.update(STATE.player.rect)
if STATE.player and not STATE.player_dead:
    # Use raw mouse position for the angle, scaled mouse is for UI
    # but since we are in GAME state, the cursor is hidden and we
    # want the mouse to feel natural.
    # Actually, get_scaled_mouse is correct for the unified screen coordinates.
    world_mx, world_my = get_scaled_mouse()
    world_mx += fcx
    world_my += fcy
    STATE.player.angle = math.atan2(
        world_my - STATE.player.rect.centery, world_mx - STATE.player.rect.centerx
    )
if STATE.melee_active:
    STATE.melee_timer -= 1
    if not STATE.melee_hit_done and STATE.melee_timer <= 10:
        STATE.melee_hit_done = True
        if STATE.player and not STATE.player_dead:
            # Shared melee resolver: range, cone, LOS, damage, particles,
            # shake are all defined in player.resolve_melee.
            landed = resolve_melee(
                STATE.player, STATE.enemies, solid_walls, C_BLOOD,
            )
            if landed:
                STATE.shots_hit += 1
                # Tal-mission: if you melee'd the commander, drop a keycard.
                # (Commander death is handled in the enemy-death loop below)
                # (value just damages.)
    if STATE.melee_timer <= 0:
        STATE.melee_active = False
        STATE.melee_hit_done = False  # reset so the next swing can hit
for b in STATE.bullets[:]:
    b["prev_x"] = b["x"]
    b["prev_y"] = b["y"]
    # Delta-time aware: dx/dy are in px/frame, so multiply by (60*dt).
    _dt_mul = 60.0 * STATE.dt
    b["x"] += b["dx"] * _dt_mul
    b["y"] += b["dy"] * _dt_mul
    b["life"] -= 1
    # --- Entity hits checked BEFORE wall hits ---
    # Previously wall collision ran first (swept clipline check), which
    # meant a bullet that reached the player's hitbox (ships radius) but
    # was also inside a cover rect was absorbed as a "wall hit" and the
    # player took no damage. Now entity hits take priority — if a bullet
    # reaches an entity, it hits that entity, not the cover.
    hit = False
    if b["owner"] == "player":
        current_window_pos = None
        if STATE.window_pos and not STATE.window_escaped:
            current_window_pos = (
                STATE.window_escape_pos
                if (STATE.firefight_triggered and STATE.window_escape_pos)
                else STATE.window_pos
            )
        if current_window_pos:
            if (
                math.hypot(
                    b["x"] - current_window_pos[0], b["y"] - current_window_pos[1]
                )
                < 22
            ):
                hit = True
                effects.burst(b["x"], b["y"], (255, 200, 100), 6, jitter=4)
                STATE.floating_texts.append(
                    [b["x"], b["y"] - 20, "NO EFFECT", (180, 180, 180), 60]
                )
        if not hit and STATE.commander_pos and not STATE.commander_escaped:
            if (
                math.hypot(b["x"] - STATE.commander_pos[0], b["y"] - STATE.commander_pos[1])
                < 22
            ):
                hit = True
                effects.burst(b["x"], b["y"], (255, 200, 100), 6, jitter=4)
                STATE.floating_texts.append(
                    [b["x"], b["y"] - 20, "NO EFFECT", (180, 180, 180), 60]
                )
        if not hit:
            for e in STATE.enemies:
                if (
                    math.hypot(b["x"] - e.rect.centerx, b["y"] - e.rect.centery)
                    < 18
                ):
                    e.hp -= b.get("damage", 34)
                    hit = True
                    STATE.shots_hit += 1
                    if STATE.player:
                        e.on_hit(STATE.player.rect.centerx, STATE.player.rect.centery)
                    else:
                        e.alerted = True
                        e.state = "CHASE"
                    # Shared burst (was 12 raw Particle()), now 10 via effects.burst
                    # to match A4's count and color: life jitter).
                    effects.burst(b["x"], b["y"], C_BLOOD, 10)
                    effects.play_impact(
                        "bullet_impact_enemy", (b["x"], b["y"]), base_volume=0.5,
                    )
                    STATE.blood_decals.append(
                        [b["x"], b["y"], 180, random.randint(8, 14)]
                    )
                    break
    if not hit and b["owner"] == "enemy" and STATE.player and not STATE.player_dead:
        if (
            math.hypot(b["x"] - STATE.player.rect.centerx, b["y"] - STATE.player.rect.centery)
            < 18
        ):
            damage = b.get("damage", ENEMY_DAMAGE)
            # BUG8 fix 4: route through take_damage() so i-frames apply.
            # If absorbed by i-frames, still mark the bullet as "hit" so
            # it gets cleaned up, but do NOT trigger the player-hit feedback
            # (no blood, no shake, no hit-stop) — the player should not
            # receive damage feedback for a hit that did no damage.
            if STATE.player.take_damage(damage):
                player_hit_this_frame = True
            hit = True
            # Shared max-clamp shake trigger (was direct assignment, which
            # could stomp a larger in-flight shake).
            effects.shake(4)
            effects.burst(b["x"], b["y"], C_BLOOD, 8)
            audio.play("bullet_impact_player", volume=0.4)
    # --- Wall hits checked AFTER (only if no entity was hit) ---
    # Same fix as enemies.py: entity hits take priority over wall hits
    # so bullets that reach the player aren't absorbed by cover rects.
    if not hit:
        hit = collide_bullet_walls(b, solid_walls)
        if hit:
            effects.burst(b["x"], b["y"], (200, 200, 140), 5, jitter=4)
            effects.play_impact("bullet_impact", (b["x"], b["y"]), base_volume=0.4)
    if (
        hit
        or b["life"] <= 0
        or not (-1000 < b["x"] < 5000 and -1000 < b["y"] < 5000)
    ):
        STATE.bullets.remove(b)
for e in STATE.enemies[:]:
    if STATE.player and not STATE.player_dead:
        e.update(
            STATE.player.rect.centerx,
            STATE.player.rect.centery,
            solid_walls,
            STATE.bullets,
            STATE.particles,
            STATE.enemies,
            STATE.fire_zones,
        )
    if e.hp <= 0:
        # Enemy death burst unified with BD (was 25 raw Particle()).
        effects.burst(e.rect.centerx, e.rect.centery, C_BLOOD, 14)
        STATE.blood_decals.append(
```

[illegible]

[illegible]